API Versioning - Complete Guide

Part 1: Why API Versioning is Required

The Problem Without Versioning

Imagine you have a mobile app that uses your API:

Version 1 (Initial):

```
GET /api/products
Response: { "id": 1, "name": "Shoes", "price": 100 }
```

Later, you change the API:

```
GET /api/products
Response: { "id": 1, "name": "Shoes", "price": 100, "discount": 10 }
```

Problem: Old mobile apps still expect the old format and will break!

Without Versioning:

- Breaking changes break all clients
- Can't evolve API safely
- Must maintain backward compatibility forever
- Risky deployments

The Solution With Versioning

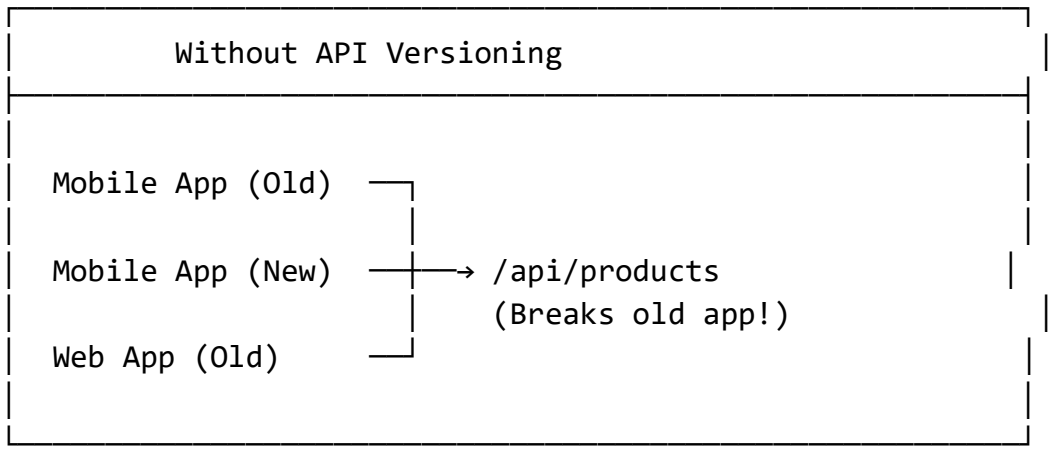
```
GET /api/v1/products  → Old format (for old apps)
GET /api/v2/products  → New format (for new apps)
```

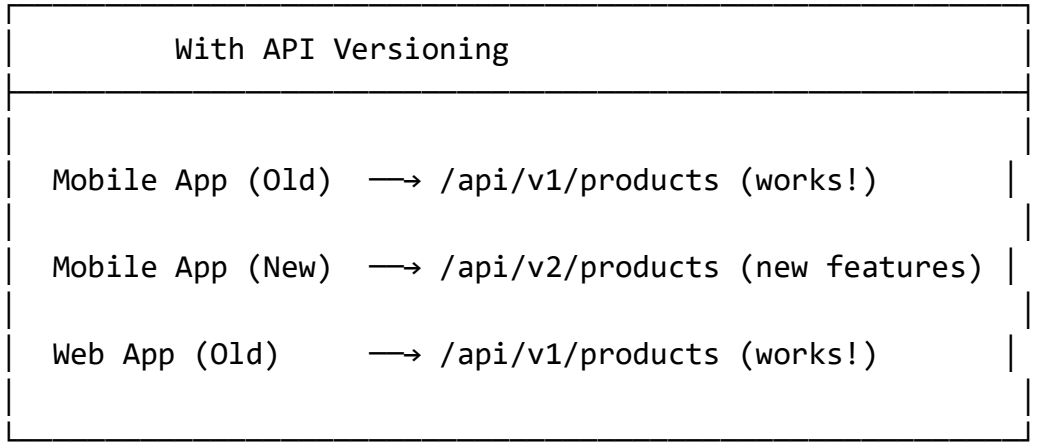
Benefits:

- Old apps continue working with v1

- New apps can use v2 features
- Safe to make breaking changes
- Gradual migration path

Real-World Example





Part 2: Your Current Project Status

Result: Your project does NOT currently use API versioning.

Your current routes (from Basket Controller):

```
[Route("api/v1/[controller]")]
public class BasketController : ControllerBase
```

```
{
    [HttpGet("{userName}")]
    public async Task<ActionResult> GetBasket(string userName)
    // ...
}
```

Note: You have v1 in the route, but this is hardcoded - not true API versioning. It's just a static string.

What you're missing:

- No API versioning package
- No versioning configuration in Program.cs
- No version attributes on controllers
- No version negotiation (URL, header, or query string)

Part 3: Step-by-Step Implementation Guide

Step 1: Install API Versioning Package

For each service (Catalog, Basket, Ordering, etc.):

```
dotnet add package Microsoft.AspNetCore.Mvc.Versioning
```

Or add to .csproj:

```
<PackageReference Include="Microsoft.AspNetCore.Mvc.Versioning"
Version="8.0.0" />
```

Step 2: Configure API Versioning in Program.cs

File: `Catalog.API/Program.cs` (add after builder.Services.AddControllers())

```
// Add API Versioning
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
```

```
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
});

// Add API Version Explorer (for Swagger)
builder.Services.AddApiVersionSetExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "Catalog API",
        Version = "v1",
        Description = "Catalog Service API v1"
    });
    options.SwaggerDoc("v2", new OpenApiInfo
    {
        Title = "Catalog API",
        Version = "v2",
        Description = "Catalog Service API v2"
    });
});
```

Configuration Options Explained:

- `DefaultApiVersion`: Default version if not specified
- `AssumeDefaultVersionWhenUnspecified`: Use default if client doesn't specify version
- `ReportApiVersions`: Return version in response headers

Step 3: Add Version Attribute to Controller

File: `Catalog.API/Controllers/ProductController.cs`

```
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/[controller]")]
[ApiController]
public class ProductController : ControllerBase
{
    [HttpGet]
```

```
    public async Task<ActionResult> GetProducts()
    {
        // v1 implementation
    }
}
```

For v2 (create new controller or add attribute):

```
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/[controller]")]
[ApiController]
public class ProductV2Controller : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult> GetProducts()
    {
        // v2 implementation with new features
    }
}
```

Step 4: Versioning Strategies

Strategy 1: URL Versioning (Most Common)

URL Format:

```
/api/v1/products
/api/v2/products
```

Controller:

```
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/[controller]")]
public class ProductController : ControllerBase
{
    // ...
```

```
}
```

Access:

- GET http://localhost:8000/api/v1/products
- GET http://localhost:8000/api/v2/products

Strategy 2: Header Versioning

Header Format:

```
api-version: 1.0
```

Controller:

```
[ApiVersion("1.0")]
[Route("api/[controller]")]
public class ProductController : ControllerBase
{
    // ...
}
```

Access:

```
GET http://localhost:8000/api/products
Headers:
  api-version: 1.0
```

Configure in Program.cs:

```
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
    options.ApiVersionReader = new HeaderApiVersionReader("api-
version");
```

```
});
```

Strategy 3: Query String Versioning

Query Format:

```
/api/products?api-version=1.0
```

Configure in Program.cs:

```
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
    options.ApiVersionReader = new QueryStringApiVersionReader("api-
version");
});
```

Step 5: Deprecate Old Versions

When you want to deprecate v1:

```
[ApiVersion("1.0", Deprecated = true)]
[Route("api/v{version:apiVersion}/[controller]")]
public class ProductController : ControllerBase
{
    // v1 is deprecated but still works
}
```

Response header will show:

```
api-deprecated-versions: 1.0
api-supported-versions: 2.0
```

Step 6: Multiple Versions in Same Controller

```
[ApiVersion("1.0")]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/[controller]")]
public class ProductController : ControllerBase
{
    [HttpGet, MapToApiVersion("1.0")]
    public async Task<ActionResult> GetProductsV1()
    {
        // v1 implementation
    }

    [HttpGet, MapToApiVersion("2.0")]
    public async Task<ActionResult> GetProductsV2()
    {
        // v2 implementation
    }
}
```

Step 7: Configure Swagger for Multiple Versions

File: Program.cs

```
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "Catalog API",
        Version = "v1",
        Description = "Catalog Service API v1"
    });
    options.SwaggerDoc("v2", new OpenApiInfo
    {
        Title = "Catalog API",
        Version = "v2",
        Description = "Catalog Service API v2"
    });
```

```
});

// In app configuration
app.UseSwagger();
app.UseSwaggerUI(options =>
{
    options.SwaggerEndpoint("/swagger/v1/swagger.json", "Catalog API
v1");
    options.SwaggerEndpoint("/swagger/v2/swagger.json", "Catalog API
v2");
});
```

Part 4: Complete Example for Your Basket Service

Update Basket.API/Program.cs

```
// Add API Versioning
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
});

builder.Services.AddApiVersionSetExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "Basket API",
        Version = "v1",
        Description = "Basket Service API v1"
    });
});
```

Update Basket.API/Controllers/BasketController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace Basket.API.Controllers
{
    [ApiVersion("1.0")]
    [Route("api/v{version:apiVersion}/[controller]")]
    [ApiController]
    public class BasketController : ControllerBase
    {
        private readonly IMediator _mediator;
        private readonly ILogger<BasketController> _logger;

        public BasketController(IMediator mediator,
ILogger<BasketController> logger)
        {
            _mediator = mediator;
            _logger = logger;
        }

        // GET: api/v1/basket/{userName}
        [HttpGet("{userName}")]
        public async Task<ActionResult<ShoppingCartDto>>
GetBasket(string userName)
        {
            var query = new GetBasketByUserNameQuery(userName);
            var result = await _mediator.Send(query);
            _logger.LogInformation("Fetching Basket for {@userName}",
userName);
            return Ok(result);
        }

        // POST: api/v1/basket
        [HttpPost]
        public async Task<ActionResult<ShoppingCartDto>>
CreateOrUpdateBasket([FromBody] CreateShoppingCartCommand
createShoppingCartCommand)
        {
```

```
            var result = await
_mediator.Send(createShoppingCartCommand);
            return Ok(result);
        }

        // DELETE: api/v1/basket/{userName}
        [HttpDelete("{userName}")]
        public async Task<IActionResult> DeleteBasket(string userName)
        {
            var cmd = new DeleteBasketByUserNameCommand(userName);
            await _mediator.Send(cmd);
            return Ok();
        }

        // Checkout
        [HttpPost("[action]")]
        public async Task<IActionResult> Checkout([FromBody]
BasketCheckoutDto dto)
        {
            await _mediator.Send(new BasketCheckoutCommand(dto));
            return Accepted();
        }
    }
}
```

Create BasketV2Controller.cs (for v2)

```
using Microsoft.AspNetCore.Mvc;

namespace Basket.API.Controllers
{
    [ApiVersion("2.0")]
    [Route("api/v{version:apiVersion}/[controller]")]
    [ApiController]
    public class BasketV2Controller : ControllerBase
    {
        private readonly IMediator _mediator;
        private readonly ILogger<BasketV2Controller> _logger;
```

```
        public BasketV2Controller(IMediator mediator,
ILogger<BasketV2Controller> logger)
        {
            _mediator = mediator;
            _logger = logger;
        }

        // GET: api/v2/basket/{userName} - with new features
        [HttpGet("{userName}")]
        public async Task<ActionResult<ShoppingCartDto>>
GetBasket(string userName)
        {
            var query = new GetBasketByUserNameQuery(userName);
            var result = await _mediator.Send(query);

            // v2: Add additional data
            _logger.LogInformation("Fetching Basket for {@userName}",
userName);
            return Ok(result);
        }

        // POST: api/v2/basket - with new validation
        [HttpPost]
        public async Task<ActionResult<ShoppingCartDto>>
CreateOrUpdateBasket([FromBody] CreateShoppingCartCommand
createShoppingCartCommand)
        {
            // v2: Add additional validation
            if (createShoppingCartCommand == null)
            {
                return BadRequest("Invalid basket data");
            }

            var result = await
_mediator.Send(createShoppingCartCommand);
            return Ok(result);
        }
    }
```

```
}
```

Part 5: Testing API Versioning

Test v1

```
curl http://localhost:8001/api/v1/basket/john
```

Test v2

```
curl http://localhost:8001/api/v2/basket/john
```

Check Response Headers

```
api-supported-versions: 1.0, 2.0
api-deprecated-versions:
```

Part 6: Best Practices

1. **Start with v1** - Always start with version 1.0
2. **Semantic Versioning** - Use MAJOR.MINOR (1.0, 1.1, 2.0)
3. **Breaking Changes = New Major Version** - 1.0 → 2.0
4. **Non-Breaking = New Minor Version** - 1.0 → 1.1
5. **Deprecate Before Removing** - Mark as deprecated, then remove later
6. **Document Changes** - Update Swagger documentation
7. **Communicate with Clients** - Notify API consumers of changes

Summary

Your Current Status: No API versioning implemented (just hardcoded v1 in routes)

To Implement:

1. Add `Microsoft.AspNetCore.Mvc.Versioning` package

2. Configure in Program.cs
3. Add `[ApiVersion]` attribute to controllers
4. Use `v{version:apiVersion}` in routes
5. Configure Swagger for multiple versions

Recommended Strategy: URL versioning (`/api/v1/`, `/api/v2/`) - most common and easy to understand

This is a network connectivity error - your connection to an external service (likely the AI service) was interrupted. This is not related to your code or the API versioning discussion.

If this persists, you may need to:

- Check your internet connection
- Try again in a moment
- Restart your IDE if needed

Would you like to continue with the API versioning implementation, or do you need help with this network issue?